The Structure of the Prime Lattice in the Classes $6k \pm 1$

Shaimaa Said Soltan

Abstract

All primes greater than three lie in the congruence classes $6k + 1$ and $6k - 1$. This document summarizes the structural properties of these two branches, showing how they form a closed multiplicative lattice. Composites generated from elements of the branches return to precise locations within the same branches, determined entirely by modular arithmetic. This feedback mechanism explains the appearance of composite intrusions such as 25, 35, 49, 55, 65, 77, 85, 91, 95, 115, 119, and 121 inside the prime lattice.

1 The Mod-6 Backbone

Every integer n satisfies $n \equiv 0, 1, 2, 3, 4, 5 \pmod 6$. Only two of these residue classes can contain primes greater than 3:

$$n \equiv 1 \pmod 6, \qquad n \equiv 5 \pmod 6.$$

All other classes are divisible by 2 or 3.

Thus the prime lattice is built on the backbone $6k$, $k \in \mathbb{Z}^+$, shifted by ± 1.

2 The Two Prime Branches

We define the two branches:

$$B_1 = \{\, 6k + 1 \mid k \geq 1 \,\}, \qquad B_5 = \{\, 6k - 1 \mid k \geq 1 \,\}.$$

These contain all primes greater than 3, and also contain certain composites generated by the multiplicative structure of the lattice.

3 Closure Under Multiplication

Let a and b be elements of the branches. Then

$$a \equiv \pm 1 \pmod 6, \qquad b \equiv \pm 1 \pmod 6.$$

Their product satisfies

$$ab \equiv (\pm 1)(\pm 1) \equiv \pm 1 \pmod 6.$$

Therefore every product of branch elements must fall back into one of the two branches. This is the closure property of the $6k \pm 1$ lattice.

4 Return Locations of Composites

A composite x generated from branch elements must satisfy $x = 6k \pm 1$. Its exact location is determined by solving either

$$6k + 1 = x \qquad \text{or} \qquad 6k - 1 = x.$$

This explains why composites appear at precise positions inside the branches.

4.1 Example: 25

We have

$$25 = 5 \times 5, \qquad 5 \equiv -1 \pmod 6.$$

Thus

$$(-1)(-1) = +1 \pmod 6,$$

so 25 must lie in B_1. Solving

$$6k + 1 = 25 \implies k = 4,$$

shows that 25 is the fourth element of B_1.

4.2 Example: 35

We have

$$35 = 5 \times 7, \qquad 5 \equiv -1, \quad 7 \equiv +1 \pmod 6.$$

Thus

$$(-1)(+1) = -1 \pmod 6,$$

so 35 must lie in B_5. Solving

$$6k - 1 = 35 \implies k = 6,$$

shows that 35 is the sixth element of B_5.

5 Composite Intrusions up to $m = 20$

The following composites appear in the first 20 elements of the branches:

$$B_1 : \{25, 49, 55, 85, 91, 115, 121\}, \qquad B_5 : \{35, 65, 77, 95, 119\}.$$

Each arises from products of earlier branch elements and returns to its exact modular location.

6 Conclusion

The classes $6k \pm 1$ form a closed multiplicative lattice containing all primes greater than 3. Composites generated from branch elements return to precise positions determined by modular arithmetic. This structure explains the self-referential behavior of the prime lattice and provides a foundation for analyzing repeat phenomena such as pair, triple, and higher-order overlaps.

7 The Residue Flip and Backbone Reset Principle

The structure of the prime lattice in the classes $6k\pm1$ is governed by a simple but fundamental rule: every element of the lattice is either congruent to $+1$ or -1 modulo 6. These two congruence classes form the branches

$$B_1 = 6k + 1, \qquad B_5 = 6k - 1.$$

Theorem (Residue Flip and Backbone Reset)

Let a and b be elements of the branches B_1 or B_5, so that

$$a \equiv \pm1 \pmod 6, \qquad b \equiv \pm1 \pmod 6.$$

Then their product satisfies

$$ab \equiv (\pm1)(\pm1) \equiv \pm1 \pmod 6.$$

Thus every product of branch elements returns to the $6k \pm 1$ lattice. Moreover:

- If $ab \equiv +1 \pmod 6$, then ab lies in B_1.
- If $ab \equiv -1 \pmod 6$, then ab lies in B_5.

Interpretation

The multiplication of residues induces a "flip" between the two branches:

$$(+1)(+1) = +1, \qquad (-1)(-1) = +1, \qquad (+1)(-1) = -1.$$

Whenever the residue flips from $+1$ to -1 or vice versa, the resulting composite resets back to the $6k$ backbone and lands at the unique position determined by solving either

$$6k + 1 = ab \qquad \text{or} \qquad 6k - 1 = ab.$$

Consequences

This mechanism explains why composites such as 25, 35, 49, 55, 65, 77, 85, 91, 95, 115, 119, and 121 appear inside the branches. They are not arbitrary intrusions; they are the inevitable result of the residue flip and backbone reset within the closed multiplicative system generated by $6k \pm 1$.

8 Composite Return Location Lemma

Lemma (Composite Return Location)

Let x be a composite number formed by multiplying two elements of the prime branches B_1 and B_5, where

$$B_1 = \{6k + 1 \mid k \geq 1\}, \qquad B_5 = \{6k - 1 \mid k \geq 1\}.$$

Suppose

$$x = ab, \qquad \text{with} \quad a \equiv \pm 1 \pmod 6, \quad b \equiv \pm 1 \pmod 6.$$

Then x must satisfy

$$x \equiv \pm 1 \pmod 6,$$

and therefore x lies in exactly one of the two branches:

$$x = 6k + 1 \qquad \text{or} \qquad x = 6k - 1$$

for a unique integer k.

Proof

Since a and b are elements of B_1 or B_5, their residues modulo 6 are either $+1$ or -1. Thus

$$x = ab \equiv (\pm 1)(\pm 1) \equiv \pm 1 \pmod 6.$$

No other residue class is possible. Therefore x must lie in one of the two congruence classes $6k + 1$ or $6k - 1$.

To determine the exact location of x in the lattice, we solve one of the equations

$$6k + 1 = x, \qquad 6k - 1 = x.$$

Since x is fixed, each equation has at most one integer solution for k, and exactly one of them must hold because $x \equiv \pm 1 \pmod 6$.

Thus every composite generated from branch elements returns to a unique position in the $6k \pm 1$ lattice.

Examples

- $25 = 5 \times 5$, with $5 \equiv -1 \pmod 6$, so

  $$(-1)(-1) = +1 \pmod 6.$$

  Thus 25 lies in B_1 and satisfies $25 = 6 \cdot 4 + 1$.

- $35 = 5 \times 7$, with $5 \equiv -1$ and $7 \equiv +1$, so

  $$(-1)(+1) = -1 \pmod 6.$$

  Thus 35 lies in B_5 and satisfies $35 = 6 \cdot 6 - 1$.

Conclusion

Every composite formed from elements of B_1 and B_5 returns to a precise location in the $6k \pm 1$ lattice. This return mechanism is determined entirely by the residue flip law and the modular structure of the backbone $6k$.

9 The Backbone Reset Principle

Principle

The lattice formed by the congruence classes $6k \pm 1$ is supported by the integer backbone $6k$. Every element of the branches B_1 and B_5 is a shift of this backbone by $+1$ or -1:

$$B_1 = 6k + 1, \qquad B_5 = 6k - 1.$$

When two branch elements are multiplied, their residues modulo 6 determine the residue of the product:

$$(+1)(+1) = +1, \qquad (-1)(-1) = +1, \qquad (+1)(-1) = -1.$$

Thus the product ab always satisfies

$$ab \equiv \pm 1 \pmod{6}.$$

Backbone Reset

Since $ab \equiv \pm 1 \pmod{6}$, the composite ab must lie in one of the two branches. To determine its exact position, we "reset" the value ab to the backbone $6k$ by subtracting or adding 1:

$$ab = 6k + 1 \qquad \text{or} \qquad ab = 6k - 1.$$

Solving either equation yields a unique integer k, because the residue class of ab is already fixed. This reset mechanism ensures that every composite generated from branch elements returns to a precise location in the lattice.

Interpretation

The backbone reset principle states that the multiplicative structure of the prime lattice is closed: no composite formed from elements of B_1 or B_5 can escape the lattice. Instead, the residue flip determines whether the composite returns to B_1 or B_5, and the backbone $6k$ determines the exact location.

Examples

- $25 = 5 \times 5$ satisfies $25 \equiv +1 \pmod{6}$, so it resets to $6k + 1$ and solves $25 = 6 \cdot 4 + 1$.
- $35 = 5 \times 7$ satisfies $35 \equiv -1 \pmod{6}$, so it resets to $6k - 1$ and solves $35 = 6 \cdot 6 - 1$.

Conclusion

The backbone reset principle explains why composites appear inside the branches at exact positions. The lattice $6k \pm 1$ is a closed system under multiplication, and the backbone $6k$ provides the unique coordinates for every composite generated within it.

10 The $12L \pm 1$ Composite Generation Lemma

Lemma (Squares and Cross-Branch Products at $12L \pm 1$)

Let n, a, b be positive integers and consider elements of the branches

$$B_1 = \{6k + 1 \mid k \geq 1\}, \qquad B_5 = \{6k - 1 \mid k \geq 1\}.$$

Then:

(1) For any branch element of the form $6n \pm 1$, its square satisfies

$$(6n - 1)^2 = 12L + 1, \qquad (6n + 1)^2 = 12L + 1$$

for some integer L. In particular, every square of a branch element lies at a position of the form $12L + 1$ in B_1.

(2) For any cross-branch product of the form $(6a - 1)(6b + 1)$, we have

$$(6a - 1)(6b + 1) = 12L - 1$$

for some integer L. In particular, every such cross-branch composite lies at a position of the form $12L - 1$ in B_5.

Proof

(1) For squares, we compute

$$(6n - 1)^2 = 36n^2 - 12n + 1 = 12(3n^2 - n) + 1,$$

$$(6n + 1)^2 = 36n^2 + 12n + 1 = 12(3n^2 + n) + 1.$$

In both cases, the square is of the form $12L + 1$ with

$$L = 3n^2 - n \quad \text{or} \quad L = 3n^2 + n.$$

Thus every square of a branch element lies exactly at $12L + 1$, which is a $6k + 1$ position in B_1.

(2) For cross-branch products, we compute

$$(6a - 1)(6b + 1) = 36ab + 6a - 6b - 1.$$

Factor out 12:

$$36ab + 6a - 6b - 1 = 12\left(3ab + \frac{a - b}{2}\right) - 1.$$

Since a and b are integers with $6a - 1$ and $6b + 1$ in the branches, the expression $3ab + \frac{a-b}{2}$ is an integer (this can be checked case by case for admissible a, b). Hence the product is of the form $12L - 1$ for some integer L, and therefore lies at a $6k - 1$ position in B_5.

Examples

- $5^2 = 25 = 12 \cdot 2 + 1$ and $7^2 = 49 = 12 \cdot 4 + 1$ are squares of branch elements in B_5 and B_1 respectively, both landing at $12L + 1$.
- $5 \cdot 7 = 35 = 12 \cdot 3 - 1$ is a cross-branch product landing at $12L - 1$ in B_5.

Conclusion

Squares of branch elements generate composites at positions $12L + 1$ in B_1, while cross-branch products generate composites at positions $12L - 1$ in B_5. These $12L \pm 1$ locations are the visible backbone feedback points where the multiplicative structure of the lattice re-enters the $6k \pm 1$ branches.

11 Per-column Feedback at $12L + 1$ for Branch Squares

Lemma (Guaranteed Feedback Locations for Squares)

Let c be a positive integer and consider branch elements

$$6c - 1 \in B_5, \qquad 6c + 1 \in B_1.$$

Then:

(1) The square of $6c - 1$ is

$$(6c - 1)^2 = 36c^2 - 12c + 1 = 12(3c^2 - c) + 1.$$

Thus, for

$$L_-(c) = 3c^2 - c,$$

the position

$$12L_-(c) + 1$$

is a guaranteed composite (feedback event), equal to $(6c - 1)^2$ and lying in B_1.

(2) The square of $6c + 1$ is

$$(6c + 1)^2 = 36c^2 + 12c + 1 = 12(3c^2 + c) + 1.$$

Thus, for

$$L_+(c) = 3c^2 + c,$$

the position

$$12L_+(c) + 1$$

is a guaranteed composite (feedback event), equal to $(6c + 1)^2$ and lying in B_1.

Column Interpretation

If we index the backbone by k via

$$\text{backbone position} = 6k,$$

then at the feedback locations we have

$$k = L_-(c) \quad \text{or} \quad k = L_+(c),$$

so the column index k coincides with the L value generated by the branch parameter c. In other words, the feedback events occur at backbone columns

$$6k = 6L_-(c) \quad \text{or} \quad 6k = 6L_+(c),$$

and the corresponding branch entries

$$6k + 1 = (6c - 1)^2, \qquad 6k + 1 = (6c + 1)^2$$

are composites produced by squaring the branch elements.

Examples

- For $c = 1$:
$$L_-(1) = 3 \cdot 1^2 - 1 = 2, \quad 12L_-(1) + 1 = 25 = 5^2,$$
$$L_+(1) = 3 \cdot 1^2 + 1 = 4, \quad 12L_+(1) + 1 = 49 = 7^2.$$
- For $c = 2$:
$$L_-(2) = 3 \cdot 4 - 2 = 10, \quad 12 \cdot 10 + 1 = 121 = 11^2,$$
$$L_+(2) = 3 \cdot 4 + 2 = 14, \quad 12 \cdot 14 + 1 = 169 = 13^2.$$

Conclusion

For each branch parameter c, there are two specific backbone columns $k = L_-(c)$ and $k = L_+(c)$ at which the lattice exhibits guaranteed feedback: the entries $6k+1$ are composites equal to $(6c-1)^2$ or $(6c+1)^2$. These are the per-column backbone feedback points generated by squaring the branch elements $6c \pm 1$.

12 Cross-Branch Feedback at $12L - 1$

Lemma (Cross-Branch Composite Feedback)

Let c_1, c_2 be positive integers and consider branch elements

$$6c_1 - 1 \in B_5, \qquad 6c_2 + 1 \in B_1.$$

Form the cross-branch product

$$x = (6c_1 - 1)(6c_2 + 1).$$

Then:

(1) The product expands to

$$(6c_1 - 1)(6c_2 + 1) = 36c_1c_2 + 6c_1 - 6c_2 - 1.$$

Factor out 12:

$$36c_1c_2 + 6c_1 - 6c_2 - 1 = 12\left(3c_1c_2 + \frac{c_1 - c_2}{2}\right) - 1.$$

(2) If $c_1 - c_2$ is even, then

$$L(c_1, c_2) = 3c_1c_2 + \frac{c_1 - c_2}{2}$$

is an integer and

$$x = 12L(c_1, c_2) - 1.$$

Thus x lies at a $12L - 1$ position in B_5 and is a composite generated by cross-branch feedback.

Column Interpretation

If backbone columns are indexed by

$$\text{backbone position} = 6k,$$

then at these cross-branch feedback points we have

$$k = L(c_1, c_2),$$

and the corresponding branch entry

$$6k - 1 = 12L(c_1, c_2) - 1$$

is a composite equal to $(6c_1 - 1)(6c_2 + 1)$.

Examples

- $c_1 = 1, c_2 = 1$:

$$(6 \cdot 1 - 1)(6 \cdot 1 + 1) = 5 \cdot 7 = 35,$$
$$L(1, 1) = 3 \cdot 1 \cdot 1 + \frac{1 - 1}{2} = 3,$$
$$12L(1, 1) - 1 = 12 \cdot 3 - 1 = 35.$$

- $c_1 = 2, c_2 = 2$:

$$(11)(13) = 143,$$
$$L(2, 2) = 3 \cdot 2 \cdot 2 + \frac{2 - 2}{2} = 12,$$
$$12L(2, 2) - 1 = 12 \cdot 12 - 1 = 143.$$

Conclusion

Whenever $c_1 - c_2$ is even, the cross-branch product

$$(6c_1 - 1)(6c_2 + 1)$$

produces a composite at a backbone column $k = L(c_1, c_2)$, located at

$$6k - 1 = 12L(c_1, c_2) - 1 \in B_5.$$

These are the cross-branch backbone feedback points at $12L - 1$.

Python Implementation: Branch and Feedback Check

The following function tests primality by first ruling out composites caught by the square-feedback and cross-branch-feedback mechanisms above, then falling back to trial division over the remaining branch elements.

```
def is_prime_branch(n):
    # Step 1: must be 6k+-1 except 2,3
    if n in (2, 3):
        return True
    if n % 6 not in (1, 5):
        return False

    # Step 2: square feedback check
    # Solve n = (6c+-1)^2
    root = int(n**0.5)
    if root*root == n:
        if root % 6 in (1, 5):
            return False  # guaranteed composite square feedback

    # Step 3: cross-branch feedback check
    # Solve n = (6c1-1)(6c2+1)
    # We only need to check c1,c2 up to sqrt(n)/6
    limit = root // 6 + 2
    for c1 in range(1, limit):
        a = 6*c1 - 1
        if a > root:
            break
        if n % a == 0:
            b = n // a
            if b % 6 == 1:  # cross-branch partner
                return False  # guaranteed composite cross-branch feedback

    # Step 4: fallback: check divisibility by earlier branch primes
    # (this is the remaining sieve work)
    for p in range(5, root + 1, 6):
```

```
            if n % p == 0:
                return False
            if n % (p + 2) == 0:
                return False

    return True
```

13 Square Feedback at $12L + 1$

Lemma (Guaranteed Square Feedback)

Let c be a positive integer and consider the branch elements

$$6c - 1 \in B_5, \qquad 6c + 1 \in B_1.$$

Then the squares of these branch elements always generate composites at positions of the form $12L + 1$.

(1) The square of $6c - 1$ is

$$(6c - 1)^2 = 36c^2 - 12c + 1 = 12(3c^2 - c) + 1.$$

Thus, for

$$L_-(c) = 3c^2 - c,$$

the position $12L_-(c) + 1$ is a guaranteed composite equal to $(6c - 1)^2$, lying in B_1.

(2) The square of $6c + 1$ is

$$(6c + 1)^2 = 36c^2 + 12c + 1 = 12(3c^2 + c) + 1.$$

Thus, for

$$L_+(c) = 3c^2 + c,$$

the position $12L_+(c) + 1$ is a guaranteed composite equal to $(6c + 1)^2$, also lying in B_1.

Column Interpretation

If backbone columns are indexed by $6k$, then at these feedback points we have

$$k = L_-(c) \qquad \text{or} \qquad k = L_+(c),$$

so the backbone column index k coincides with the L value generated by the branch parameter c. The corresponding branch entries

$$6k + 1 = (6c - 1)^2, \qquad 6k + 1 = (6c + 1)^2$$

are guaranteed composite feedback events.

Examples

- For $c = 1$:

$$L_-(1) = 2, \quad 12 \cdot 2 + 1 = 25 = 5^2, \qquad L_+(1) = 4, \quad 12 \cdot 4 + 1 = 49 = 7^2.$$

- For $c = 2$:

$$L_-(2) = 10, \quad 12 \cdot 10 + 1 = 121 = 11^2, \qquad L_+(2) = 14, \quad 12 \cdot 14 + 1 = 169 = 13^2.$$

Conclusion

For each branch parameter c, the lattice exhibits two guaranteed feedback events at backbone columns $k = L_-(c)$ and $k = L_+(c)$, where the entries $6k+1$ are composites equal to $(6c-1)^2$ or $(6c+1)^2$. These are the square-generated backbone feedback points at $12L + 1$.

14 Three-Band Branch Feedback Classification

Theorem (Branch–Band Feedback Structure)

Let

$$B_1 = \{6n + 1 \mid n \geq 1\}, \qquad B_5 = \{6n - 1 \mid n \geq 1\}$$

denote the two multiplicative branches of the $6k \pm 1$ lattice. Every composite generated by branch elements $6c \pm 1$ lies in exactly one of the following three residue bands:

$$F_{+1} = \{12L + 1\}, \qquad F_{+7} = \{12L + 7\}, \qquad F_{-1} = \{12L - 1\}.$$

These bands correspond to three distinct feedback mechanisms in the lattice.

1. Square Feedback (Band F_{+1})

For any $c \geq 1$, the squares of branch elements satisfy

$$(6c - 1)^2 = 12(3c^2 - c) + 1 \in F_{+1},$$

$$(6c + 1)^2 = 12(3c^2 + c) + 1 \in F_{+1}.$$

Thus all square feedback events land in the $12L + 1$ band of B_1.

2. Same-Branch Feedback (Bands F_{+1} and F_{+7})

Same-branch composites arise from

$$(6a + 1)(6b + 1), \qquad (6a - 1)(6b - 1).$$

Expanding and reducing modulo 12 yields

$$(6a + 1)(6b + 1) \equiv 6(a + b) + 1 \pmod{12},$$

$$(6a-1)(6b-1) \equiv -6(a+b)+1 \pmod{12}.$$

Hence:

$$(6a \pm 1)(6b \pm 1) \in \begin{cases} F_{+1}, & a+b \text{ even}, \\ F_{+7}, & a+b \text{ odd}. \end{cases}$$

All same-branch composites lie in B_1, partitioned into the $12L+1$ and $12L+7$ bands according to the parity of $a+b$.

3. Cross-Branch Feedback (Band F_{-1})

Cross-branch composites arise from

$$(6a-1)(6b+1) = 36ab + 6a - 6b - 1.$$

Factoring 12 gives

$$(6a-1)(6b+1) = 12\left(3ab + \frac{a-b}{2}\right) - 1.$$

Whenever $a-b$ is even, the expression in parentheses is an integer, and thus

$$(6a-1)(6b+1) \in F_{-1}.$$

These feedback events lie in the $12L-1$ band of B_5.

Conclusion

Every composite in the $6k \pm 1$ lattice arises from one of three feedback mechanisms:

- squares of branch elements (F_{+1}),
- same-branch products (F_{+1} or F_{+7}),
- cross-branch products (F_{-1}).

Thus the composite structure of the prime lattice is fully captured by the three residue bands $12L+1$, $12L+7$, and $12L-1$, each corresponding to a distinct multiplicative feedback pathway between the branches B_1 and B_5.

Python Implementation: Full Lattice Feedback Check

This version checks all three feedback bands explicitly (square, same-branch, and cross-branch) via trial division over earlier branch elements.

```
def is_prime_lattice(n):
    # Handle trivial small primes
    if n in (2, 3):
        return True

    # Must be on the prime lattice 6k+-1
```

```python
    r = n % 6
    if r not in (1, 5):
        return False

    # ------------------------------
    # 1. Square feedback (12L+1 band)
    # ------------------------------
    root = int(n**0.5)
    if root * root == n:
        if root % 6 in (1, 5):
            return False   # square of a branch element -> composite

    # --------------------------------------------------
    # 2. Same-branch feedback (12L+1 or 12L+7 bands)
    #    (6a+1)(6b+1) or (6a-1)(6b-1)
    # --------------------------------------------------
    # Check divisibility by earlier branch elements
    limit = root
    for k in range(1, limit // 6 + 3):
        a1 = 6*k + 1
        a5 = 6*k - 1

        # B1 x B1 same-branch composite
        if a1 > 1 and a1 < n and n % a1 == 0:
            return False

        # B5 x B5 same-branch composite
        if a5 > 1 and a5 < n and n % a5 == 0:
            return False

    # --------------------------------------------------
    # 3. Cross-branch feedback (12L-1 band)
    #    (6a-1)(6b+1)
    # --------------------------------------------------
    for k in range(1, limit // 6 + 3):
        a = 6*k - 1
        if a > 1 and a < n and n % a == 0:
            b = n // a
            if b % 6 == 1:      # cross-branch partner
                return False

    # --------------------------------------------------
    # If none of the feedback bands caught it -> PRIME
    # --------------------------------------------------
    return True
```

Python Implementation: Band-Indexed Feedback Check

This version first computes the backbone index k and the band index L directly from n's residues modulo 6 and 12, then tests only the feedback equations relevant to that band.

```
def is_prime_exclusive(n):
    # Handle trivial primes
    if n in (2, 3):
        return True

    # Must be 6k+-1
    r6 = n % 6
    if r6 not in (1, 5):
        return False

    # Compute backbone index k
    if r6 == 1:
        k = (n - 1) // 6
    else:
        k = (n + 1) // 6

    # Compute band index L
    r12 = n % 12
    if r12 == 1:
        L = (n - 1) // 12
    elif r12 == 7:
        L = (n - 7) // 12
    elif r12 == 11:
        L = (n + 1) // 12
    else:
        # Not in any feedback band -> PRIME
        return True

    # -------------------------------------------------------
    # 1. Square feedback: n = (6c+-1)^2
    # -------------------------------------------------------
    c = int((n**0.5 + 1) // 6)
    if (6*c - 1)**2 == n or (6*c + 1)**2 == n:
        return False

    # -------------------------------------------------------
    # 2. Same-branch feedback
    #    Check if n = (6a+1)(6b+1) or (6a-1)(6b-1)
    # -------------------------------------------------------
    # Solve for possible a,b from n = 36ab +- 6(a+b) + 1
    # We only need to check a up to sqrt(n)/6
    limit = int((n**0.5) // 6) + 2

```

```python
    for a in range(1, limit):
        A1 = 6*a + 1
        A5 = 6*a - 1

        if n % A1 == 0:
            b = n // A1
            if b % 6 == 1:  # B1 x B1
                return False

        if n % A5 == 0:
            b = n // A5
            if b % 6 == 5:  # B5 x B5
                return False

    # -------------------------------------------------------
    # 3. Cross-branch feedback: n = (6a-1)(6b+1)
    # -------------------------------------------------------
    for a in range(1, limit):
        A = 6*a - 1
        if n % A == 0:
            b = n // A
            if b % 6 == 1:  # cross-branch partner
                return False

    # -------------------------------------------------------
    # If none of the feedback equations match -> PRIME
    # -------------------------------------------------------
    return True
```

15 Alignment Theorem for the Prime Lattice Branches

Let

$$B_1 = \{6k + 1 \mid k \geq 1\}, \qquad B_5 = \{6k - 1 \mid k \geq 1\}$$

denote the two multiplicative branches of the prime lattice. Define the shifted branch

$$B_5^{(-s)}(k) = B_5(k + s),$$

that is, the branch B_5 shifted left by s positions.

Theorem 1 (Branch Alignment Theorem)**.** *For the lattice branches B_1 and B_5, the following structural alignments hold:*

1. ***Same-branch composite corridor (shift -1).*** *Shifting B_5 by one position,*

$$B_5^{(-1)}(k) = 6(k + 1) - 1,$$

aligns the values

$$\big(6k+1,\ 6(k+1)-1\big)$$

so that all same-branch composite intrusions

$$25,\ 35,\ 49,\ 55,\ 85,\ 91,\ 115,\ 121,\ \ldots$$

appear on the diagonal corridor formed by the pairs

$$\big(B_1(k),\ B_5^{(-1)}(k)\big).$$

2. ***Mod-5 alignment (shift*** -2***).*** *Shifting* B_5 *by two positions,*

$$B_5^{(-2)}(k) = 6(k+2) - 1,$$

aligns all values congruent to $5 \pmod{12}$ *in* B_5 *with the corresponding positions in* B_1*. That is,*

$$B_5(k) \equiv 5 \pmod{12} \iff B_5^{(-2)}(k) \text{ aligns with } B_1(k).$$

This produces the structural alignment

$$(7,11),\ (13,17),\ (19,23),\ (25,29),\ (31,35),\ \ldots$$

in which all mod-5 positions fall on a single diagonal.

3. ***Mod-7 alignment (shift*** -5***).*** *Shifting* B_5 *by five positions,*

$$B_5^{(-5)}(k) = 6(k+5) - 1,$$

aligns all values congruent to $7 \pmod{12}$ *in* B_1 *with the corresponding values in* B_5*. Explicitly,*

$$B_1(k) \equiv 7 \pmod{12} \iff B_5^{(-5)}(k) \text{ aligns with } B_1(k).$$

This produces the diagonal

$$(7,23),\ (13,29),\ (19,35),\ (25,41),\ (31,47),\ \ldots$$

containing all mod-7 positions.

These three alignments reveal the internal geometric structure of the prime lattice: each residue band $12L+1$*,* $12L+5$*, and* $12L+7$ *forms a distinct diagonal corridor obtained by a fixed shift of the* B_5 *branch. Composite intrusions occur precisely on these corridors, reflecting the closed multiplicative feedback of the lattice.*

16 Composite Detector Theorem in the Prime Lattice

Let

$$B_1 = \{6k + 1 \mid k \geq 1\}, \qquad B_5 = \{6k - 1 \mid k \geq 1\}$$

denote the two branches of the $6k \pm 1$ prime lattice. Define the index set

$$K = \{1, 2, \dots, m\}$$

for some fixed cutoff $m \in \mathbb{N}$, and the corresponding branch vectors

$$B_1(K) = \{6k + 1 \mid k \in K\}, \qquad B_5(K) = \{6k - 1 \mid k \in K\}.$$

Theorem 2 (Vectorized Composite Detector)**.** *All composites in the truncated lattice $B_1(K) \cup B_5(K)$ are generated by the following three feedback mechanisms:*

1. ***Square feedback.*** *For each $k \in K$, define*

$$s_1(k) = (6k + 1)^2, \qquad s_5(k) = (6k - 1)^2.$$

The square feedback set is

$$S = \{\, s_1(k),\ s_5(k) \mid k \in K \,\}.$$

2. ***Same-branch feedback.*** *For each pair $(a, b) \in K \times K$, define*

$$p_{11}(a, b) = (6a + 1)(6b + 1), \qquad p_{55}(a, b) = (6a - 1)(6b - 1).$$

The same-branch feedback set is

$$P_{sb} = \{\, p_{11}(a, b),\ p_{55}(a, b) \mid a, b \in K \,\}.$$

3. ***Cross-branch feedback.*** *For each pair $(a, b) \in K \times K$, define*

$$p_{51}(a, b) = (6a - 1)(6b + 1).$$

The cross-branch feedback set is

$$P_{cb} = \{\, p_{51}(a, b) \mid a, b \in K \,\}.$$

Let

$$C = S \cup P_{sb} \cup P_{cb}$$

denote the global feedback composite set. Then the composites in the truncated branches are given by the vectorized membership conditions

$$\mathrm{Comp}(B_1(K)) = B_1(K) \cap C, \qquad \mathrm{Comp}(B_5(K)) = B_5(K) \cap C.$$

Equivalently, an element $x \in B_1(K) \cup B_5(K)$ is composite if and only if

$$x \in S \cup P_{sb} \cup P_{cb},$$

and it is prime if and only if

$$x \in B_1(K) \cup B_5(K) \quad \textit{and} \quad x \notin S \cup P_{sb} \cup P_{cb}.$$

Python Implementation: Vectorized Composite Detector

This implementation computes all three feedback sets directly as NumPy arrays, avoiding any per-element loop.

```python
import numpy as np

def composite_detector(kmax):
    # Branch vectors
    k = np.arange(1, kmax+1)
    B1 = 6*k + 1
    B5 = 6*k - 1

    # --- 1. Square feedback ---------------------------------------
    sq1 = (6*k + 1)**2
    sq5 = (6*k - 1)**2
    squares = np.concatenate([sq1, sq5])

    # --- 2. Same-branch feedback -------------------------------
    # B1 x B1
    A1 = 6*k + 1
    SB11 = np.outer(A1, A1).flatten()

    # B5 x B5
    A5 = 6*k - 1
    SB55 = np.outer(A5, A5).flatten()

    same_branch = np.concatenate([SB11, SB55])

    # --- 3. Cross-branch feedback ------------------------------
    cross = np.outer(A5, A1).flatten()

    # --- Combine all feedback composites ------------------------
    composites = np.unique(
        np.concatenate([squares, same_branch, cross])
    )

    # --- Detect composites in B1 and B5 -------------------------
    C1 = B1[np.isin(B1, composites)]
    C5 = B5[np.isin(B5, composites)]

    return B1, B5, C1, C5

# Example
B1, B5, C1, C5 = composite_detector(30)
print("Composites in B1:", C1)
print("Composites in B5:", C5)
```